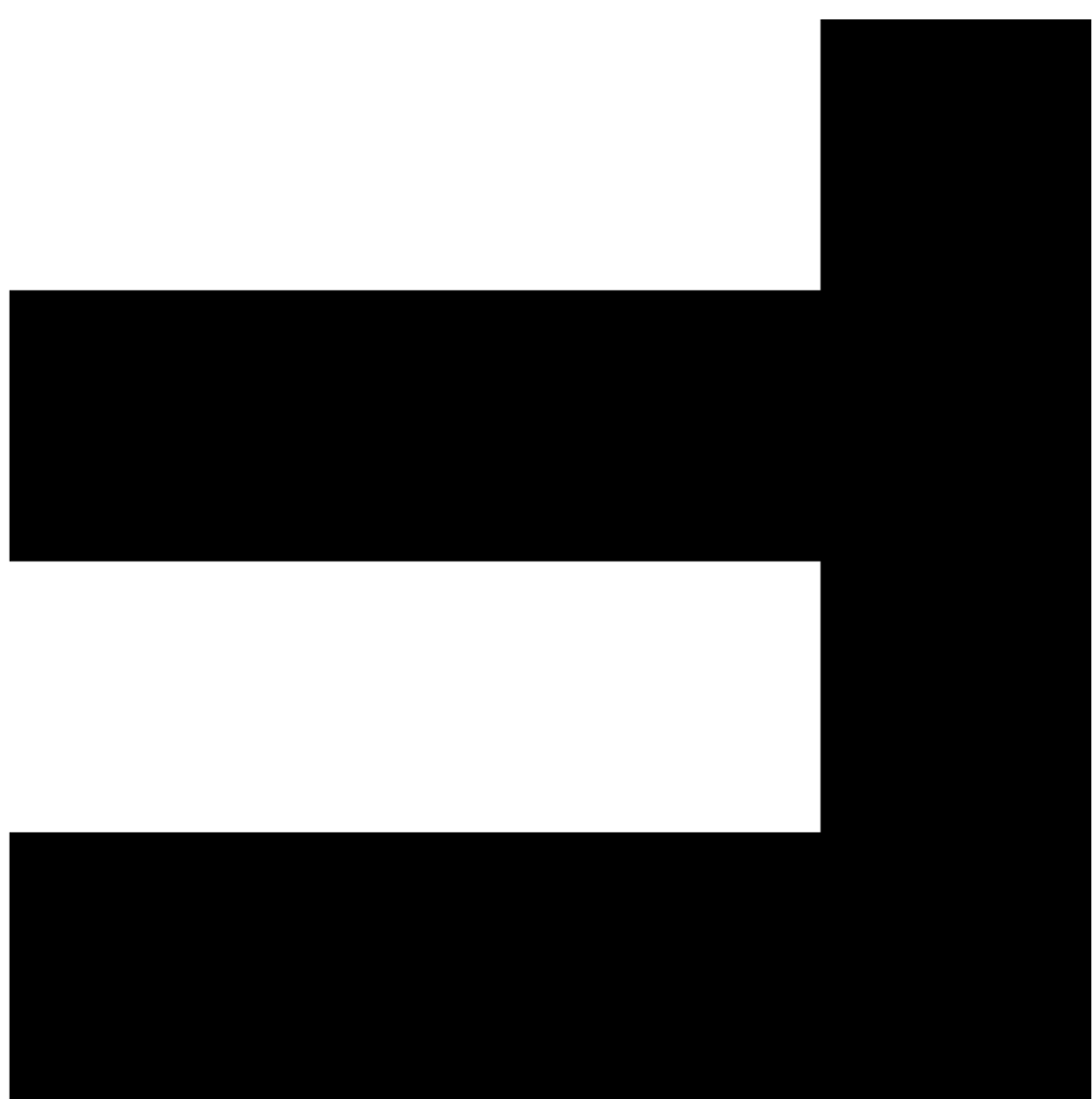


Ports of Call: Ein microservice-basierter Wirtschaftssimulator

Ein Wirtschaftsklassiker der 90er



Motivation

Dieses Projekt entstand im Rahmen des Kurses „Enterprise Applications“ im Studiengang Software and Information Engineering an der FH Vorarlberg. Ziel war die Modernisierung des klassischen Wirtschaftssimulators „Ports of Call“ als webbasierte, verteilte Mehrspieler-Anwendung - eine komplexe, zustandsbehaftete Domäne aus Handel, Logistik, Risiko und Wettbewerb.



Abb 1. Startbildschirm zum Erstellen oder Beitreten einer Spielsitzung: Reedereiname festlegen oder über den Session-Code einer anderen Sitzung beitreten

Spielprinzip

Jeder Spieler leitet eine Reederei, kauft Frachtschiffe, nimmt Frachtaufträge zwischen Häfen an und plant Routen. Herzstück ist ein serverautoritatives Tick-System: Ein zentraler Scheduler treibt die Zeit voran – ein Tick entspricht einem Spieltag und wird standardmäßig alle 5 Sekunden ausgelöst. Pro Tick bewegen sich Schiffe einen Wegpunkt weiter, Treibstoff wird verbraucht, Märkte werden aktualisiert und Zufallsereignisse ausgewürfelt. Über WebSockets bleiben alle Clients in Echtzeit synchron; Sieger ist, wer am Session-Ende das Leaderboard anführt.

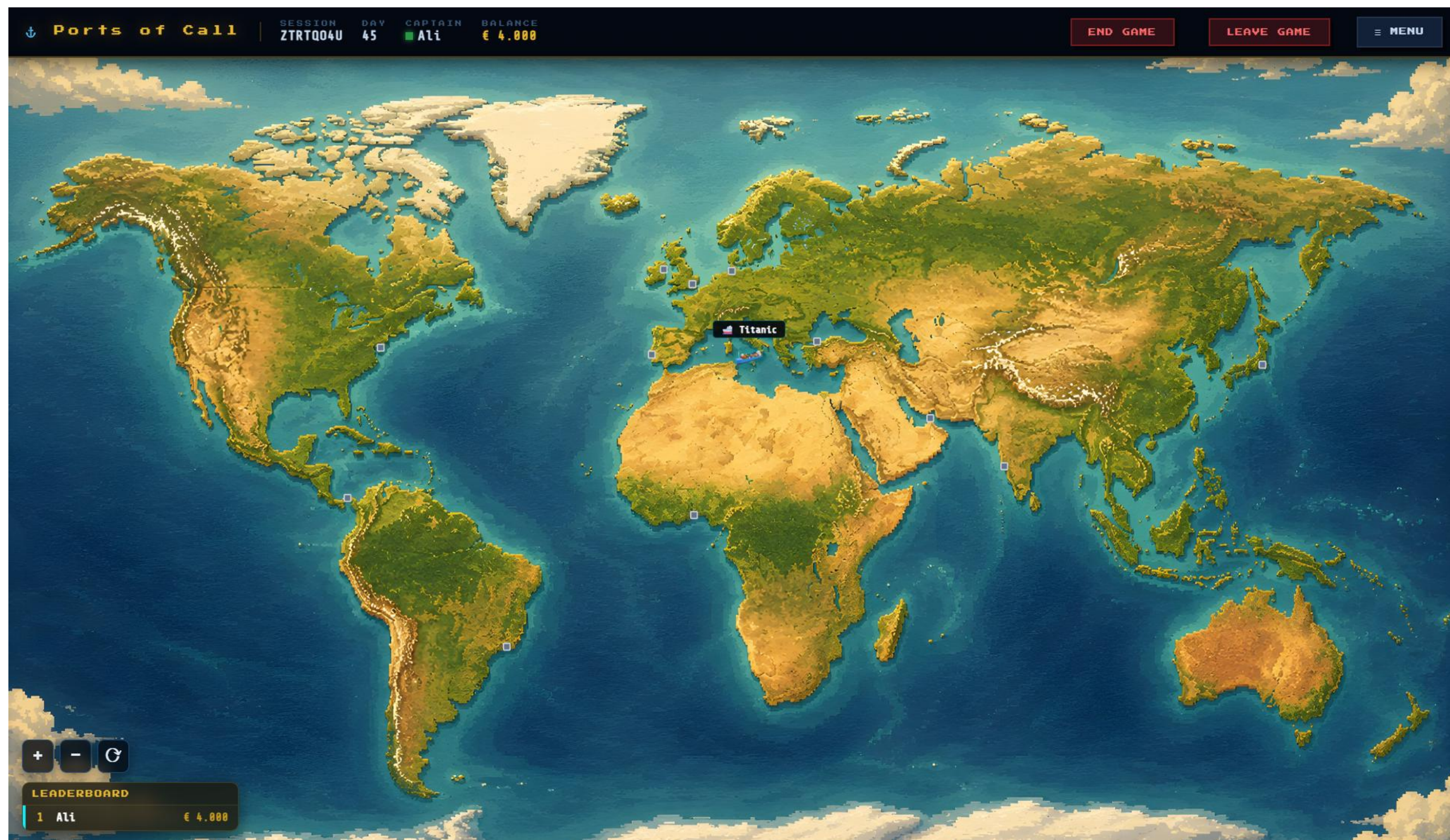


Abb 2. Weltkarte als dauerhafter Spielhintergrund mit allen Häfen und den in Echtzeit fahrenden Schiffen

Systemarchitektur

Um den Anforderungen einer modernen Enterprise-Applikation zu entsprechen, wurde das System als entkoppelte Microservice-Architektur realisiert – mit klarer Domänentrennung (Bounded Contexts), unabhängigen Datenbanken und ausschließlich azyklischen Abhängigkeiten.

- API-Gateway (Spring Cloud Gateway): einziger öffentlicher Eingang; Routing, CORS und WebSocket-Tunnel.
- service-session: Lobby, Session-Start, Tick-Zähler und End-Game-Vote.
- service-player: Konto, Position, Heimathafen sowie Bank, Kredite und Hypotheken.
- service-freight: Schiffe, Frachtverträge, Zoll und Schmuggel.
- service-navigation: Häfen, Routen, Wegpunkte und politische Ereignisse.
- service-game-engine: stateless Orchestrator, der jeden Tick koordiniert und den GameState broadcastet.
- Datenbank: vier eigenständig PostgreSQL-Datenbanken (Database-per-Service), Referenzen nur per ID

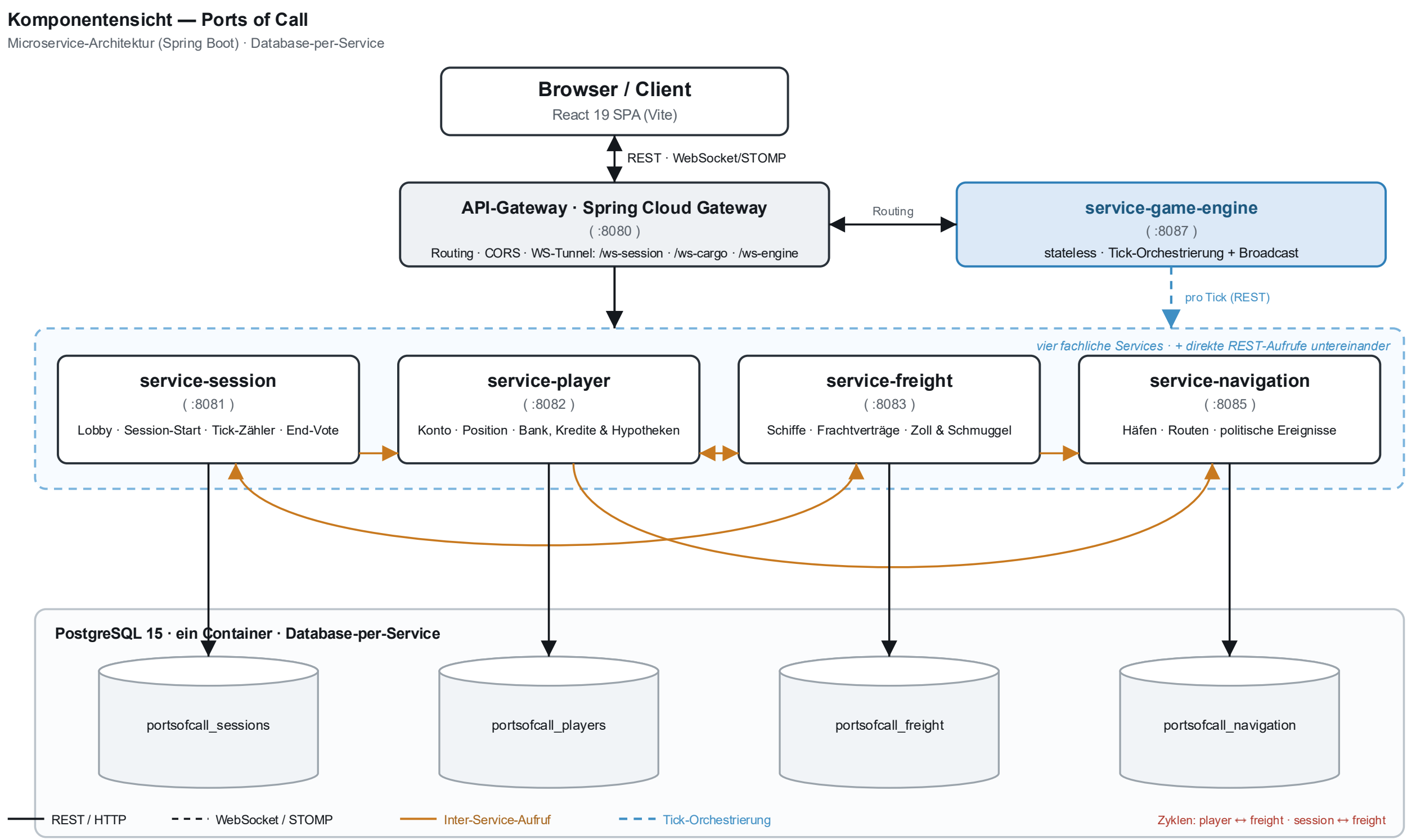


Abb 3. Komponentensicht der verteilten Architektur: fünf Spring-Boot-Services hinter einem API-Gateway, Echtzeit-Kommunikation über WebSocket/STOMP und vier PostgreSQL-Datenbanken nach dem Prinzip Database-per-Service

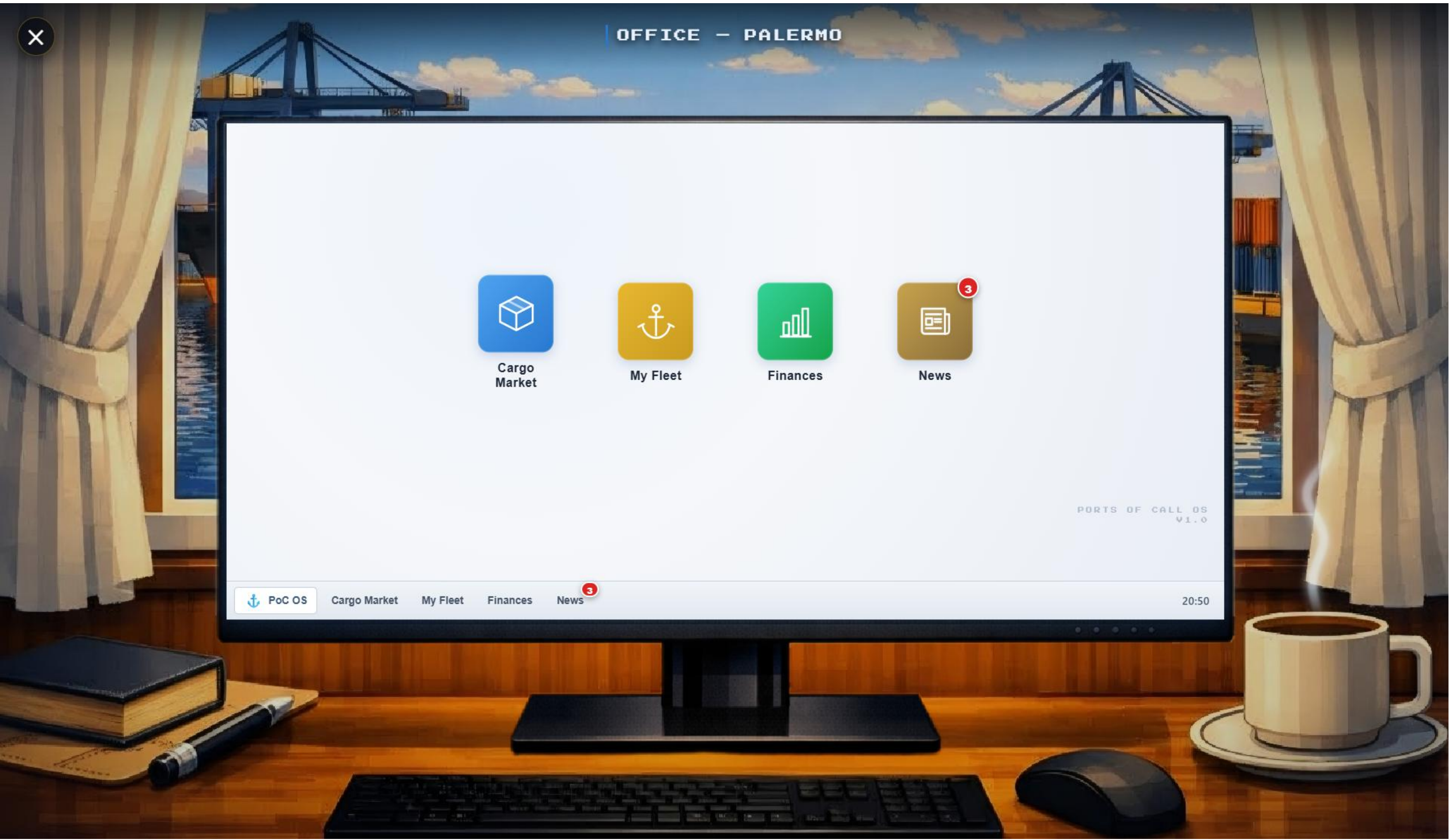


Abb 4. Office-Ansicht der Reederei: Überblick aktuelle Frachtaufträge, Flotte, Finanzen und Neuigkeiten über aktuelle Weltgeschehnisse im Spiel, alles als eine eigene App-Komponente dargestellt

